



Observabilidade com OpenTelemetry

Conceito e ferramentas

Introdução

Em situações com arquiteturas complexas e distribuídas, garantir a confiabilidade, performance e segurança dessas aplicações tornou-se fundamental. Aqui entra o conceito de "observabilidade" – uma evolução do monitoramento tradicional que se refere à capacidade de entender o estado interno de um sistema a partir de seus dados externos, como métricas, rastreio(traces) e logs.

Neste contexto, o **OpenTelemetry** surge como um projeto que visa oferecer APIs, bibliotecas e instrumentação padrão para coleta de dados de rastreio e métricas. É uma fusão de dois grandes projetos de observabilidade, o OpenTracing e o OpenCensus, reunindo o melhor de ambos para fornecer uma única solução para instrumentação de código. Com isso, desenvolvedores/DevOps têm ferramentas mais consistentes e integradas para coletar, enviar e analisar dados observáveis.

São três os **pilares** da observabilidade:

Métricas: São medidas quantitativas que fornecem uma visão em alto nível do comportamento e performance do sistema. Pode-se pensar nelas como a "pulsção" ou "temperatura" do sistema.

Rastreios (Traces): Proporcionam uma visão detalhada de como as transações se movem através de todos os componentes e serviços de uma aplicação, ajudando a identificar gargalos, falhas ou ineficiências.

Logs: São registros detalhados de eventos, que fornecem um histórico de ações e erros. Os logs ajudam a diagnosticar problemas específicos, oferecendo um registro imutável dos eventos à medida que ocorrem.

Vários "backends" ou plataformas de observabilidade oferecem suporte a esses três pilares, incluindo soluções populares como **Prometheus** (focado em métricas), **Jaeger** e **Zipkin** (para rastreio) e **Elasticsearch** (para log e busca). Muitos destes sistemas agora oferecem suporte nativo ou integrações com OpenTelemetry, tornando mais fácil do que nunca coletar, armazenar e analisar dados observáveis.

Em resumo, a observabilidade garante a confiabilidade e desempenho de sistemas em um cenário de arquiteturas cada vez mais complexas. Com ferramentas como OpenTelemetry e backends especialistas, as equipes podem obter insights mais profundos, identificar problemas rapidamente e garantir que seus sistemas funcionem de maneira eficiente e confiável.

Instrumentação

Adicionar Dependências

Via NuGet, adicione as bibliotecas necessárias ao seu projeto, como ``OpenTelemetry``, ``OpenTelemetry.Extensions.Hosting`` e ``OpenTelemetry.Exporter.OpenTelemetryProtocol``. Pode variar de



Configuração Inicial

.NET 6 ou superior

No ponto de entrada da aplicação (por exemplo, `Startup.cs`), podemos adicionar:

```
1  var resourceBuild = ResourceBuilder.CreateDefault()
2      .AddService(serviceName: "ons.sager.apuracao", serviceVersion:
3  "1.0.0").AddTelemetrySdk();
4
5  builder.Services.AddOpenTelemetry()
6      .WithMetrics(builder =>
7      {
8          builder.AddMeter("ons.sager.apuracao")
9              .SetResourceBuilder(resourceBuild)
10             .AddHttpClientInstrumentation()
11             .AddAspNetCoreInstrumentation()
12             .AddOtlpExporter(exporter => { exporter.Protocol =
13 OpenTelemetry.Exporter.OtlpExportProtocol.Grpc; });
14         })
15         .WithTracing(builder =>
16         {
17             builder.AddSource("ons.sager.apuracao")
18                 .SetResourceBuilder(resourceBuild)
19                 .AddHttpClientInstrumentation(options => options.RecordException =
20 true)
21                 .AddAspNetCoreInstrumentation(options => options.RecordException =
22 true)
23                 .SetErrorStatusOnException()
24                 .AddOtlpExporter(exporter =>
25                 {
26                     exporter.Protocol =
27 OpenTelemetry.Exporter.OtlpExportProtocol.Grpc;
28                 });
29             });
30
31
32 builder.Logging.ClearProviders()
33     .AddOpenTelemetry(loggerOptions =>
34     {
35         loggerOptions
36             .SetResourceBuilder(resourceBuild)
37             .AddOtlpExporter(exporter =>
38             {
39                 exporter.Protocol =
40 OpenTelemetry.Exporter.OtlpExportProtocol.Grpc;
```



```

        loggerOptions.IncludeFormattedMessage = true;
        loggerOptions.IncludeScopes = true;
        loggerOptions.ParseStateValues = true;
    });

```

O SDK do OpenTelemetry irá ler a variável `OTEL_EXPORTER_OTLP_ENDPOINT`, que será o endpoint do collector para onde serão enviados os dados. Para aplicações no OpenShift, podemos utilizar a porta GRPC do service do OpenTelemetry (:4317). Para aplicações hospedadas no IIS, por enquanto, devemos utilizar o protocolo HttpProtobuf, sendo necessário criar uma rota no OpenShift expondo a porta :4318 do serviço do collector. O valor da `OTEL_EXPORTER_OTLP_ENDPOINT` com a rota deverá ser colocado nas variáveis de ambiente do Windows e realizar um `iisreset`, se necessário.

.NET 4.8

Segue guia para configuração simplificada para aplicações em `NET_Framework`, usando o componente `POPProvider`:

[POP_Provider com OpenTelemetry](#)

Segue o abaixo a forma customizada de realizar as configurações diretamente na aplicação em `NETFramework` sem uso do `POP_Provider`:

No ponto de entrada da aplicação (por exemplo, `Global.asax.cs`), podemos adicionar:

```

1  _tracerProvider = Sdk.CreateTracerProviderBuilder()
2  .AddHttpClientInstrumentation()
3  .AddAspNetInstrumentation((options) => options.Enrich
4      = (activity, eventName, rawObject) =>
5      {
6          if (eventName.Equals("OnStopActivity")) //Workaround: No .net 4.8 o
7          nome registrado no tracing é registrado pelo OTel incorretamente.
8          {
9              if (rawObject is HttpResponseMessage httpResponse)
10             {
11                 activity.SetTag("responseType", httpResponse.ContentType);
12
13                 if (activity.Tags.Where(x => x.Key == "http.target").Any())
14                 {
15                     activity.DisplayName = activity.Tags.Where(x => x.Key ==
16                     "http.target").First().Value;
17

```



```
20     }
21   })
22   .AddSource("meu-servico-com-opentelemetry")
23   .AddOtlpExporter(exporter => { exporter.Protocol =
24   OpenTelemetry.Exporter.OtlpExportProtocol.HttpProtobuf; })
25   .SetResourceBuilder(
26     ResourceBuilder.CreateDefault()
27     .AddService(serviceName: "meu-servico-com-opentelemetry",
28     serviceVersion: "1.0.0"))
29
30   .Build();
31
32
33   _meterProvider = Sdk.CreateMeterProviderBuilder()
34     .AddMeter("meu-servico-com-opentelemetry")
35     .AddHttpClientInstrumentation()
36     .AddAspNetInstrumentation()
37
38     .AddOtlpExporter(exporter => { exporter.Protocol =
39     OpenTelemetry.Exporter.OtlpExportProtocol.HttpProtobuf; })
40     .SetResourceBuilder(
41       ResourceBuilder.CreateDefault()
42       .AddService(serviceName: "meu-servico-com-opentelemetry",
43       serviceVersion: "1.0.0"))
44     .Build();
```

Note que algumas bibliotecas do pacote *Instrumentation*, como por exemplo a

OpenTelemetry.Instrumentation.AspNet ainda não tem uma versão final. Isto se deve a algumas convenções semânticas ainda não terem o consenso da comunidade e colaboradores, o que impede a liberação de uma versão estável geral (GA). Mas podemos usar sua RC sem problemas.

Sobre Logs: Como pressupõe-se que a instrumentação do OpenTelemetry em uma aplicação 4.8 será em uma aplicação mais antiga e que já deva ter sido instrumentada com Log4Net, sugere-se o uso de um appender para direcionar os logs para o collector do OpenTelemetry, assim mantendo a correlação, como no exemplo abaixo:

```
1 public class OpenTelemetryAppender : AppenderSkeleton
2 {
3     Microsoft.Extensions.Logging.ILogger _logger =
4     Logger.Factory().CreateLogger<OpenTelemetryAppender>();
5
6     protected override void Append(LoggingEvent loggingEvent)
7     {
8         LogLevel logLevel = ConverterParaLogLevel(loggingEvent.Level);
9         _logger.Log(logLevel, loggingEvent.ExceptionObject,
10         loggingEvent.RenderedMessage);
```



```
13     private LogLevel ConverterParaLogLevel(Level level)
14     {
15         if (level == Level.Debug)
16             return LogLevel.Debug;
17         else if (level == Level.Info)
18             return LogLevel.Information;
19         else if (level == Level.Warn)
20             return LogLevel.Warning;
21         else if (level == Level.Error)
22             return LogLevel.Error;
23         else if (level == Level.Fatal)
24             return LogLevel.Critical;
25         else
26             return LogLevel.None;
27     }
28 }
29
30 public class Logger
31 {
32     public static ILoggerFactory Factory() { return _loggerFactory; }
33
34     private static ILoggerFactory _loggerFactory =
35     LoggerFactory.Create(builder =>
36     {
37         builder.AddOpenTelemetry(options =>
38         {
39             options.SetResourceBuilder(ResourceBuilder.CreateDefault()
40                 .AddService(serviceName: "meu-servico-com-opentelemetry",
41 serviceVersion: "1.0.0"));
42
43             options.AddOtlpExporter(otlpOptions =>
44             {
45                 otlpOptions.Protocol =
46 OpenTelemetry.Exporter.OtlpExportProtocol.HttpProtobuf;
47
48                 });
49             });
50         });
51     });
52 }
```

Ignorando determinadas rotas

Algumas rotas, como a de AngularSettings podem ser desnecessárias de ter traces registrados, sendo possível ignorar tais rotas e evitando lixo nos traces. Nas options dentro de AddAspNetInstrumentation, podemos colocar como a seguir:



```
2 | 1
3 |     options.Filter = (httpContext) =>
4 |     {
5 |         return
6 |         httpContext.Request.Url.AbsolutePath.Contains("AngularSettings") is false;
7 |     };
   | })
```

Assim, registramos tudo, menos os requests à rota que possua AngularSettings no seu path.

Instrumentação Automática

Após incluir e configurar as bibliotecas de instrumentação, frameworks como .NET e a classe HttpClient serão automaticamente instrumentados. Também é possível uma instrumentação no-code, [aqui](#)  é possível encontrar mais de detalhes de como fazer em .NET.

Instrumentação Manual

A instrumentação manual, que é a mais indicada para melhor resultado, é feita com o próprio SDK do .NET, utilizando Activity para trace, ILogger para logs e Meter para métricas. Exemplos de instrumentação podem ser encontrados no repositório de exemplo [OTel - Repos \(ons.org.br\)](https://github.com/ons.org.br/OTel-Repos)  ou, no próprio repositório do OpenTelemetry <https://github.com/open-telemetry/opentelemetry-dotnet/tree/main/examples/MicroserviceExample>.

Exportação via OTLP

Como visto no código da configuração inicial, vamos utilizar uma camada de abstração para onde enviaremos os dados instrumentados. Essa abstração é um serviço do próprio OpenTelemetry conhecido como Collector. Os dados coletados na aplicação serão automaticamente enviados ao collector para o devido tratamento e envio aos backends de preferência.

Collector

O **OpenTelemetry Collector** é um componente da arquitetura do OpenTelemetry. Ele atua como um intermediário que pode receber, processar e exportar dados de telemetria, proporcionando uma camada de abstração entre os instrumentos de sua aplicação (onde os dados são gerados) e os backends de observabilidade (onde os dados são analisados e armazenados).

Receivers (Receptores):

São os pontos de entrada para os dados. Os receivers aceitam dados de várias fontes, geralmente em diferentes formatos. Exemplos de receivers incluem:

- **otlp** (OpenTelemetry Protocol)



- **prometheus**

Processors (Processadores):

Depois que os dados são recebidos, eles passam por uma série de processors. Estes são usados para transformar, enriquecer, filtrar ou modificar os dados antes de serem exportados. Alguns exemplos de processors são:

- **batch** : Agrupa vários spans/traces em lotes para otimizar o envio.
- **queued_retry** : Implementa uma tentativa de reenvio para casos em que a exportação falha inicialmente.
- **memory_limiter** : Limita a quantidade de memória que o collector pode usar, descartando dados se necessário.
- **attributes** : Adiciona, modifica ou exclui atributos dos spans.

Exporters (Exportadores):

Uma vez processados, os dados estão prontos para serem enviados aos backends de observabilidade. Exporters são responsáveis por essa tarefa. Dependendo de para onde você deseja enviar seus dados, você configurará um ou mais exporters. Exemplos de exporters incluem:

- **otlp**
- **jaeger**
- **zipkin**
- **prometheus**
- **kafka**
- **elastic**

Extensions (Extensões):

As extensões são componentes opcionais que podem ser usados para adicionar funcionalidades ou recursos ao collector, mas que não se encaixam diretamente como receivers, processors ou exporters. Exemplos podem incluir:

- **health_check** : Fornece um endpoint para verificar a saúde do collector.
- **pprof** : Adiciona a capacidade de capturar perfis do collector, útil para depuração.
- **zpages** : Fornece um UI para ver traces e estatísticas do collector.



Para instalar o OpenTelemetry no seu projeto do OpenShift, é necessário somente ir ao catálogo e procurar por “OpenTelemetry Collector” e em seguida instalar. O operador sobe uma imagem padrão do Collector, caso necessário o uso da distribuição ‘contrib’, é necessário somente alterar o atributo 'image' para algo como otel/opentelemetry-collector-contrib:0.xx.0, por exemplo. A utilização da distribuição contrib se faz necessário para uso de exporters como o do Elastic.

Configuração:

OpenTelemetry

Para definir como o Collector deve operar, você fornecerá um arquivo de configuração (geralmente em YAML). Neste arquivo, você especificará quais receivers devem ser ativados, como os dados devem ser processados (e em que ordem), e para onde os dados devem ser exportados.

Um exemplo básico de configuração pode se parecer com isto:

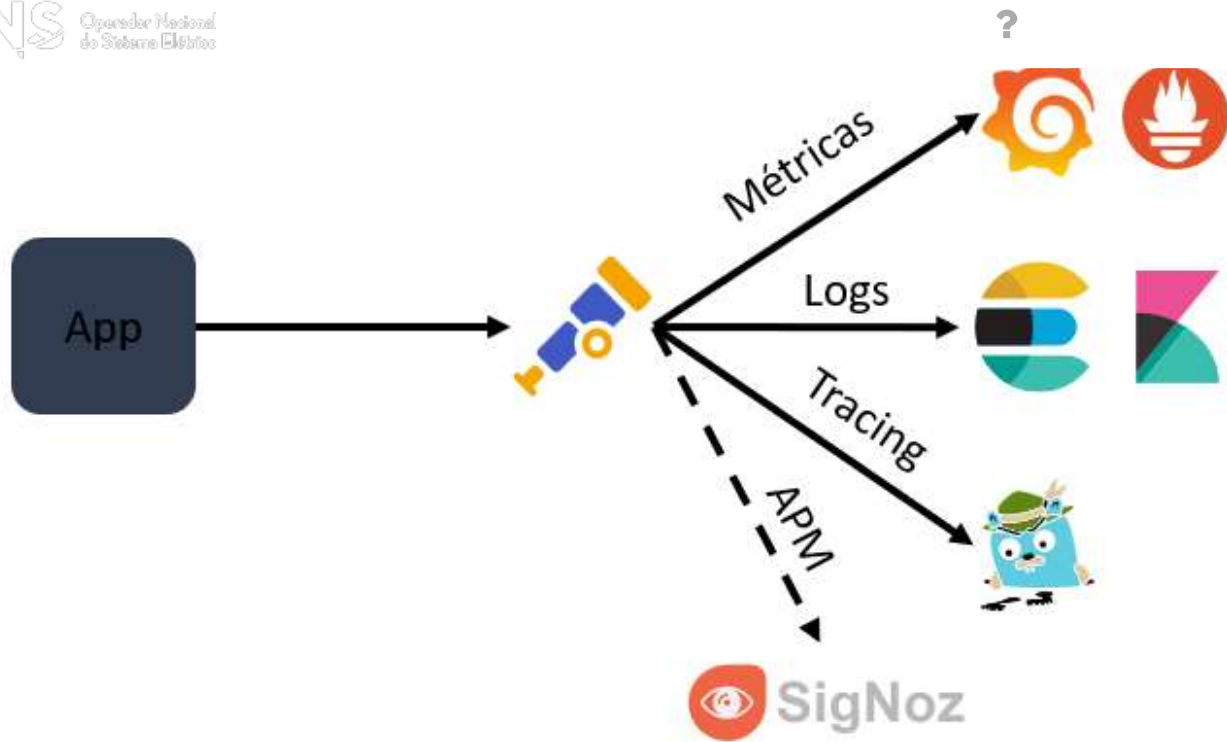
```
1 spec:
2   image: >=
3     image-registry.openshift-image-
4 registry.svc:5000/openshift/opentelemetry-collector:0.84.0
5   config: |
6     receivers:
7       otlp:
8         protocols:
9           grpc:
10            http:
11
12     exporters:
13       logging:
14         loglevel: debug
15
16       otlp:
17         endpoint: <svc do jaeger dev|tst|hmg|prd>:4317
18         tls:
19           insecure: true
20
21       elasticsearch/log:
22         endpoints: [http://<host do ambiente>:9200]
23         logs_index: logs-<comunidade>-<aplicacao>-<ambiente>
24         api_key: <api key gerada no elastic>
25         tls:
26           insecure: false
27           insecure_skip_verify: true
28
29       prometheusremotewrite:
30         endpoint: http://<host do ambiente>:9090/api/v1/write
31         target_info:
32           enabled: true
```




```
35     batch:
36
37     connectors:
38         spanmetrics:
39
40     service:
41         pipelines:
42             traces:
43                 receivers: [otlp]
44                 processors: [batch]
45                 exporters: [logging, otlp, spanmetrics]
46             logs:
47                 receivers: [otlp]
48                 processors: [batch]
49                 exporters: [elasticsearch/log]
50             metrics:
51                 receivers: [otlp, spanmetrics]
52                 processors: [batch]
53                 exporters: [logging, prometheusremotewrite]
54     resources:
55         limits:
56             cpu: 500m
57             memory: 256Mi
58         requests:
59             cpu: 250m
60             memory: 128Mi
61     autoscaler:
62         maxReplicas: 10
63         minReplicas: 1
64         targetCPUUtilization: 80
        targetMemoryUtilization: 80
```

Arquitetura adotada para uso do OpenTelemetry

Novas aplicações nascem com essa nova arquitetura, e como mencionado acima, utilizaremos o OpenTelemetry como um intermediário entre as aplicações e os backends de observabilidade, gerando uma camada de abstração entre aplicações e ferramentas de trace, log e métricas.



Arquitetura proposta para o uso de OpenTelemetry

Ambientes

Kibana, Grafana

DEV/TST - 10.211.254.59 (<https://observabilidade-dev.ons.org.br>) "... /kibana" ou "... /grafana"

HMG - 10.212.254.8 (<https://observabilidade-hom.ons.org.br>) "... /kibana" ou "... /grafana"

PRD - 10.213.254.25 (<https://observabilidade-prd.ons.org.br>) "... /kibana" ou "... /grafana"

ELASTIC - Porta 9200

KIBANA - Porta 5601

GRAFANA - Porta 3000

PROMETHEUS - Porta 9464 e 9090

Jaeger

Produção:

Interface: <https://jaeger-shared-tools.apps.ocpp.ons.org.br>

Service: jaeger-collector-headless.shared-tools.svc.cluster.local

Homologação :

Interface: <https://jaeger-shared-tools-hom.apps.ocpp.ons.org.br>

**Teste :**

Interface: `https://jaeger-shared-tools.apps.ocpd.ons.org.br`

Service: `jaeger-collector-headless.shared-tools.svc.cluster.local`

Desenvolvimento :

Interface: `https://jaeger-shared-tools-dev.apps.ocpd.ons.org.br`

Service: `jaeger-collector-headless.shared-tools-dev.svc.cluster.local`

Acessos

Para criação de usuários de consulta às ferramentas, Api Keys para integração com os backends e outras dúvidas, podem procurar a equipe de DevOps.

Trilha de Observabilidade

Foi criada uma trilha dividida em 3 partes falando de conceitos, ferramentas e colocando um pouco a mão em código:

[Trilha de Observabilidade](#) 

Distribuído por [Wiki.js](#)